다중 선택
다중 선택 1

10장 비전 트랜스포머

10.0 개요

비전 트랜스포머의 등장

- **ViT(Vision Transformer):** 2020년 구글 논문 *"An Image is Worth 16×16 Words: Transformers for Image Recognition at Scale"* 에서 소개
- 이미지를 **자연어 처리 방식처럼** 분류해보려는 시도에서 탄생
- CNN의 합성곱 계층 ✕ → 트랜스포머의 **셀프 어텐션** 적용
  - CNN: **지역 특징**을 추출
  - ViT: 셀프 어텐션으로 **전체 이미지를 한 번에 처리**
- **ViT의 한계:** 패치가 왼쪽→오른쪽, 위→아래로 **순차 입력** → 2차원 구조의 이미지 특성을 온전히 반영한다고 할 수 없음
- **해결을 위해 제안된 모델:** 물체의 크기나 해상도를 **계층적으로 학습**
  - **Swin Transformer**
  - **CvT (Convolutional Vision Transformer)**

모델	핵심 아이디어
Swin Transformer	**로컬 윈도(Local Window)** 로 계층마다 어텐션 되는 패치의 크기·개수를 다양하게 구성. **상대적 위치 편향** 반영
CvT	기존 **합성곱 연산을 ViT에 적용**, 저수준·고수준 특징을 계층적으로 반영. Q, K, V 중 **K, V를 축소**해 계산 복잡도 ↓

- CvT 예시: 얼굴의 **저수준 특징** = 눈·코·입 같은 작은 단위 / **고수준 특징** = 눈·코·입을 포함한 전체 얼굴
- 실습 데이터: **FashionMNIST** (10개 클래스: T-Shirt, Trouser, Pullover, Dress, Coat, Sandal, Shirt, Sneaker, Bag, Ankle Boot)

10.1 ViT

- 이전 컴퓨터비전 연구: CNN에 트랜스포머의 **셀프 어텐션 모듈**을 적용한 모델이 많음
- **ViT는 트랜스포머 구조 자체를 컴퓨터비전에 적용한 첫 번째 연구**

BERT vs ViT

- 둘 다 트랜스포머 구조를 그대로 사용하지만 **입력 데이터를 만드는 과정**이 다름

구분	BERT	ViT
입력 단위	문장보다 작은 단위의 **토큰**	격자로 나뉜 작은 단위의 **이미지 패치**

구분	BERT	ViT
입력 방식	순차적으로 입력	순차적으로 입력 (왼쪽→오른쪽, 위→아래 시퀀셜 배열 가정)

1) 합성곱 모델과 ViT 모델 비교

- 목적은 같음: **이미지 특징을 잘 표현하는 임베딩** 만들기
- 과정이 다름

구분	CNN 임베딩	ViT 임베딩
학습 방식	이미지 패치 중 **일부만 선택**하여 학습	이미지를 작은 패치로 나눠 **패치 간 상관관계** 학습
예시 (고양이 눈)	눈 부분 패치(1, 2, 4, 5)만 학습, 나머지(3, 6, 7, 8, 9)는 관여 ×	**셀프 어텐션**으로 **모든 패치**가 학습에 관여
수용 영역 (RF)	좁음 → 전체 이미지 표현에 **수많은 계층** 필요	**어텐션 거리(Attention Distance)** 계산 → **ViT 레이어 1개**로도 전체 이미지 정보 표현 가능

- ViT의 장점: **패치 단위**로 처리(CNN은 픽셀 단위) → **더 작은 모델로도 높은 성능**
- ViT의 단점
  - **입력 이미지 크기가 고정** → 크기가 다른 이미지는 **크기 맞추는 전처리** 필요
  - CNN은 공간적 위치 정보를 고려하지만, ViT는 **패치 간 상대적 위치 정보만** 고려 → **이미지 변환에 취약**할 수 있음

2) ViT의 귀납적 편향

💡 **귀납적 편향 (Inductive Bias)**
: 일반화 성능 향상을 위한 **모델의 가정(Assumption)**

- 가정이 올바르면 **적은 데이터로도 높은 일반화 성능**
- 너무 강하면 **다른 유형의 데이터·관계를 표현하기 어려움**

딥러닝 모델	귀납적 편향	이유
CNN	**지역적(Local)** 편향	이미지를 작은 조각으로 나눠 각 조각의 특징 추출 후 조합 → 이미지 데이터에 강함
RNN	**순차적(Sequential)** 편향	이전 시간의 상태를 기억하고 현재 입력과 결합 → 시계열 데이터에 강함
ViT	**거의 없음**	Q, K, V 임베딩으로 **일반화된 관계**를 학습 → 대용량 데이터에서 이미지 특징 간 관계를 잘 학습

3) ViT 모델 구조

- 구성: **패치 임베딩(Patch Embedding) + 인코더(Encoder) 계층**
- 패치 임베딩과 인코더로 특징 추출 → 분류/회귀 등 작업에 맞는 출력으로 변환

패치 임베딩

: 입력 이미지를 **작은 패치로 분할**하는 과정

1. **이미지 크기 전처리:** 정방형으로 조절 (예: 640×480 → 224×224)

2. **패치 분할: 합성곱 계층**을 활용해 시퀀셜 배열 생성
    - **커널 크기(Kernel Size)** = 패치 크기, **간격(Stride)** = 패치 이동 폭 (둘 다 하이퍼파라미터)
    - 패치 개수(가로 또는 세로) = (image size − kernel size) / stride + 1
    - 예) 이미지 9×9, 커널 3×3, 간격 3 → (9−3)/3+1 = 3 → 가로 3 × 세로 3 = **9개**
    - 예) 커널 3×3, 간격 1 → (9−3+1) × (9−3+1) = **49개**
3. **분류 토큰** `[CLS]` **추가:** 패치 배열 **가장 왼쪽**에 추가 → **전체 이미지를 대표하는 벡터**
4. **위치 임베딩(Position Embedding):** 패치 위치 정보를 임베딩 벡터로 변환 → 기존 패치 벡터에 **더함** (인접 패치 간 관계 학습)
5. **계층 정규화** 적용 → 패치 임베딩 완성

- 이미지를 패치로 나눠 처리하면 **GPU 메모리 한계 극복**, 더 큰 이미지 처리 가능

인코더 계층

- 각 패치 벡터를 인코더에 입력 → 다양한 Q, K, V 임베딩의 관계 학습 (7장 **멀티 헤드 어텐션**과 동일)
- 인코더 구성: 계층 정규화 → 셀프 어텐션 → 벡터 더하기 & 정규화 → 순방향 신경망 → 벡터 더하기 & 정규화 (**N번 반복**)
- 마지막 레이어에서 **분류 토큰([CLS])의 특징 벡터** 추출
    - 시퀀셜 산출물 전체 ×, **[CLS] 벡터만** 사용해서 분류
    - [CLS] 벡터를 **순방향 신경망(완전 연결 계층)** 에 연결 → 분류 학습

4) 모델 실습 (ViT 미세 조정)

- **허깅 페이스** + **FashionMNIST** 사용
    - FashionMNIST: 의류 이미지, **10개 클래스, 훈련 60,000개 / 테스트 10,000개** (MNIST보다 복잡한 문제용)
    - 실습용 샘플링: **훈련 10,000개 (클래스당 1,000) / 테스트 1,000개 (클래스당 100)**

학습 흐름

1. **데이터 불러오기 + 균등 샘플링:** `torchvision.datasets.FashionMNIST`, `defaultdict`로 클래스별 인덱스 → `Subset`
2. **이미지 전처리:** `AutoImageProcessor` (사전 학습 모델 `google/vit-base-patch16-224-in21k`: ImageNet-21 기반, 224×224, 패치 16)
    - `ToTensor` → `Resize(224×224)` → `Lambda` **(단일 채널을 3채널로 복제)** → `Normalize`
3. **데이터로더 +** `collate_fn` **:** ViT 입력은 **딕셔너리** `{"pixel_values", "labels"}` 형태 → 집합 함수로 미니 배치 병합
    - `pixel_values`: (배치, 채널, 높이, 너비) = (32, 3, 224, 224)
4. **사전 학습 모델 로드:** `ViTForImageClassification`
    - `num_labels`, `id2label`, `label2id`, `ignore_mismatched_sizes=True`로 **미세 조정** (분류기 `out_features` = 10)
5. **학습:** `TrainingArguments` + `Trainer`

6. **평가: 매크로 평균 F1 점수 + 혼동 행렬**

패치 임베딩 확인

- 패치 임베딩 = **16×16 커널, 16 간격 Conv2d**
- 224×224 이미지 → (224−16)/16+1 = **14** → 14×14 = **196개** 패치 (채널 768)
- 앞에 **[CLS] 토큰** 추가 → **197개** 벡터

단계	shape
입력 이미지	[32, 3, 224, 224]
패치 임베딩	[32, 196, 768]
[CLS] + 패치 임베딩	[32, 197, 768]

TrainingArguments 주요 매개변수

매개변수	의미
`output_dir`	체크포인트 저장 경로
`save_strategy`	체크포인트 저장 간격 (no / steps / epoch)
`evaluation_strategy`	평가 간격 (no / steps / epoch)
`learning_rate`	초기 학습률
`per_device_train/eval_batch_size`	학습/평가 배치 크기
`num_train_epochs`	학습 반복 수
`weight_decay`	가중치 감소
`load_best_model_at_end`	최상의 모델 선택 여부
`metric_for_best_model`	최상의 모델 선정 기준 (accuracy, precision, f1 등)
`logging_dir` / `logging_steps`	로그 저장 폴더 / 출력 간격
`seed`	무작위 시드
`fp16`	Mixed precision 사용 여부 (일부 파라미터 32bit 유지 + 일부 16bit → 메모리 ↓, 더 큰 배치 가능)

성능 평가 지표

- **매크로 평균 F1 점수(Macro Average F1-Score):** 다중 클래스이므로 모든 클래스의 F1 점수를 평균
- `compute_metrics`: 예측값 `argmax` → `evaluate.load("f1")` → `average="macro"`

학습 결과 (ViT)

Epoch	Training Loss	Validation Loss	F1-Score
1	0.7062	0.6377	0.8905
2	0.4954	0.4781	0.9166
3	0.4078	0.4357	0.9231

- 테스트 정확도 **약 92%**
- **혼동 행렬 분석:** '셔츠'를 '티셔츠'나 '코트'로 **오분류**하는 경우가 상대적으로 많음

- 모델 개선 방법: **하이퍼파라미터 조정, 모델 구조 변경, 전처리 개선, 데이터 증강**

10.2 Swin Transformer

등장 배경

- 기존 트랜스포머 기반 모델의 문제
  - **고정된 패치 크기** → 세밀한 예측이 필요한 **의미론적 분할(Semantic Segmentation)** 등에 부적합
  - 셀프 어텐션이 입력 이미지 크기에 대해 **2차(Quadratic) 계산 복잡도** → 고해상도 처리 어려움
- **해결: 계층적(hierarchy) 특징 맵** 구성 → **1차(Linear) 계산 복잡도**
- 2021년 발표, 이미지 분류·객체 감지·영상 분할 등에서 강력한 성능 → **백본 모델**로 사용
- **시프트 윈도(Shifted Window)** 기술: 입력 이미지를 패치로 분할 → 패치를 쌓아 **윈도** 구성 → **윈도 영역만** 셀프 어텐션 계산

1) ViT와 Swin Transformer 차이

- ViT의 한계
  - 획일적인 패치 크기·위치 제약 → **다양한 이미지 크기·종횡비**에 적용 어려움
  - 패치 단위 입력 → 공간 정보가 완전히 보존되지 않을 수 있음
  - 패치 수 ↑ → 계산량 ↑ → **대규모 데이터셋에서 컴퓨팅 자원 많이 소모**
- Swin: **로컬 윈도(Local Window)** (입력 이미지 처리에 쓰는 **고정 크기의 작은 윈도**)로 물체의 크기·해상도를 계층적으로 학습
  - **ViT보다 높은 정확도 + 적은 매개변수**, 더 넓은 범위의 이미지 크기·종횡비 처리
- ViT는 계층마다 어텐션 패치 크기·개수 **동일**, Swin은 **계층적으로 다양**
- **로컬 윈도**가 각 패치의 **세부 정보**에 집중 + **시프트 윈도**로 **전역 특징** 확인 → 고해상도 이미지 효율적 처리

구분	ViT	Swin Transformer
분류 헤드	[CLS] 토큰 사용	**각 토큰의 평균값** 사용
로컬 윈도	×	○
상대적 위치 편향	×	○
계층별 패치 크기	동일	다양

2) Swin Transformer 모델 구조

- 흐름: **패치 파티션** → **선형 임베딩** → **Swin Transformer 블록** → **패치 병합** (반복)
  1. 입력 이미지를 일정 크기 패치로 분할 → **선형 임베딩**으로 고정 차원 벡터 변환
  2. **Swin Transformer 블록** (어텐션 + 계층 정규화 + MLP) → 패치 간 상호작용
  3. **패치 병합**으로 합치며 전체 이미지 정보 추출 → 분류
- **4개의 스테이지**, 각 스테이지마다 해상도·패치 크기가 다름
  - 스테이지 1: 선형 임베딩 → 블록
  - 스테이지 2, 3, 4: **패치 병합** → 블록

- 블록 반복 횟수: ×2, ×2, ×6, ×2

패치 파티션 / 패치 병합

- **패치 파티션(Patch Partition):** 입력 이미지를 작은 사각형 패치로 분할
  - 공간 정보 보존 + 트랜스포머 계산 효율성 ↑
  - 이미지 텐서 [C, H, W]에 **로컬 윈도(크기 M)** 추가
  - 예) [3, 32, 32] → 패치 가로·세로(P) = 4
- **패치 병합(Patch Merging):** 인접 패치 정보를 **저차원으로 축소** (매개변수 수 ↓)
  - [C, H, W] → **[2C, H/2, W/2]** 로 재정렬 → 증가한 채널을 저차원으로 임베딩
  - 예) [3, 8, 8] → [6, 4, 4] → 다시 3차원으로 임베딩 → [3, 4, 4]

Swin Transformer 블록

- 구성: **계층 정규화 → W-MSA → 계층 정규화 → MLP → 계층 정규화 → SW-MSA → 계층 정규화 → MLP** (순차 수행)
- 기존 MSA 대신 **W-MSA와 SW-MSA**를 사용

구분	W-MSA (Window MSA)	SW-MSA (Shifted Window MSA)
방식	특징 맵을 **겹치지 않게** 윈도로 나눠 각 윈도에서 **독립적으로** 셀프 어텐션	윈도를 **가로/세로로 이동**시킨 후 **인접 윈도 간** 셀프 어텐션
장점	지역 특징 분석, 셀프 어텐션 비용 **2차 → 1차 복잡도**로 감소	**윈도 사이의 연결성** 구축 → 윈도 간 관계 정보 추출
단점	**윈도 내부 패치에만** 어텐션 수행	윈도 개수가 많아 연산량 ↑ → **순환 시프트 + 어텐션 마스크**로 해결

- **W-MSA 효율성: 효율적인 배치 계산(Efficient Batch Computation)** 으로 연산량 대폭 감소 (로컬 윈도 연산에도 배치 축 사용 가능)

> 💡 **순환 시프트 (Cyclic Shift) & 어텐션 마스크**
>
> - **순환 시프트:** 윈도 이동 시 이동한 위치의 정보를 **이전 위치에서 가져옴**, 윈도 크기 M보다 작은 **M/2** 만큼 이동
> - **어텐션 마스크:** 이동된 위치에서 **불필요한 연산을 하지 않도록** 방지
> - **역순환 시프트(Reverse Cyclic Shift):** 셀프 어텐션 수행 후 **원래 윈도 구조로 복원**
> - 효과: 9개 윈도 파티션에 9번 어텐션 → **4개 윈도 파티션의 4번 어텐션**만으로 계산 가능

상대적 위치 편향 (Relative Position Bias)

- 기존 ViT와 다른 점: 셀프 어텐션에 **로컬 윈도 안 패치 간의 상대적 거리**를 임베딩하여 반영
- 패치 간 상대 거리를 **X축(가로 방향)** 과 **Y축(세로 방향)** 두 가지로 표기
  - X축 위치 행렬: 같은 위치 0, 아래로 떨어져 있으면 1, 위로 떨어져 있으면 −1
  - Y축 위치 행렬: 같은 위치 0, 오른쪽으로 떨어져 있으면 −1, 왼쪽으로 떨어져 있으면 1

- **계산 과정 (window_size = 2 예시)**
  1. `meshgrid`로 격자 좌표 생성 → `stack` → `flatten`
  2. 각 위치 인덱스의 **차이** 계산 → relative_coords (X, Y 위치 행렬)
  3. 각 좌표에 **window_size − 1** 더하기 (음수 제거)
  4. x_coords에 **2 × window_size − 1** 곱하기
  5. x_coords + y_coords → **상대적 위치 인덱스**
  6. `relative_position_bias_table`(크기 $(2\cdot ws-1)^2 \times num_heads$)에서 인덱스로 편향 값 조회 → [$ws^2$, ws^2, heads]로 재구성
- **셀프 어텐션 수식 (상대적 위치 편향 B 반영)**
  - $\text{Attention}(Q,K,V) = \text{SoftMax}(QK^T/\sqrt{d} + B)V$
  - Q, K, V: 쿼리·키·값 벡터 / d: 임베딩 차원

3) 모델 실습 (Swin 미세 조정)

- 사전 학습 모델: `microsoft/swin-tiny-patch4-window7-224` (ImageNet-1k, 224×224, **패치 4×4, 로컬 윈도 7×7**)
- 데이터셋 코드는 ViT 실습과 동일, **사전 학습 모델·출력 디렉터리만 변경**

모델 구조 확인

- 크게 **swin** + **classifier**
  - swin: 패치 파티션 + 선형 임베딩 → **4개 스테이지(0~3)** → 계층 정규화(layernorm) → 풀링(pooler)
  - 패치 임베딩: **4×4 커널, 4 간격** Conv2d → (224−4)/4+1 = **56** → 56×56 = **3,136개** 패치
- 각 스테이지 = **blocks**(Swin Transformer 블록) + **downsample**(패치 병합)
  - 블록 = **SwinLayer-0 (W-MSA)** + **SwinLayer-1 (SW-MSA)**
  - SwinLayer: `layernorm_before` → `attention` → `layernorm_after` → `intermediate` + `output` (MLP)
- **MLP:** 선형 변환 2개 + **GELU** 활성화 함수
  - **GELU(Gaussian Error Linear Unit):** ReLU와 유사, 가우시안 분포의 **누적 분포 함수**, 입력이 0 주변에서 **부드럽게 변화** → ReLU보다 **안정적 학습**

단계	shape
입력 이미지	[32, 3, 224, 224]
패치 임베딩	[32, 3136, 96]
W-MSA 출력	[32, 3136, 96]
SW-MSA 출력	[32, 3136, 96]
패치 병합 후	[32, 784, 192]
마지막 스테이지 후	[N, 49(7×7), 768]

- ViT는 196개 패치/768채널 + [CLS] = 197개, Swin은 **3,136개 패치/96채널, [CLS] 토큰 없이 토큰 평균값**으로 분류
  - 이유: 객체 검출처럼 여러 물체가 있는 경우 [CLS] 하나로는 모든 물체 검출이 어려움

패치 병합 연산

1. 3,136(56×56)개 패치 → **784(28×28)개**로 분할 (reduction)
2. 4개로 나뉜 텐서 (N, 28, 28, 96)을 **채널로 병합** → 96×4 = **384**
3. **선형 변환**으로 채널 절반으로 조정 (384 → **192**)
4. 결과: [N, 784, 192]

- 스테이지가 지날수록 **패치 수는 줄고, 토큰 차원은 2배씩 증가**
- 최종 특징 맵 → 풀링 계층 → 클래스 예측

학습 결과 (Swin)

Epoch	Training Loss	Validation Loss	F1
1	0.3720	0.3046	0.8985
2	0.2579	0.2688	0.9138
3	0.2607	0.2416	0.9159

- 테스트 정확도 **약 91.6%**
- ViT와 비교: **F1은 거의 동일**하지만 **검증 손실 0.4357 → 0.2416**으로 대폭 감소
- '셔츠'를 '티셔츠', '코트'로 **오분류하는 문제 완화**, **수렴 속도도 더 빠름**

10.3 CvT (Convolutional Vision Transformer)

- **합성곱 신경망의 계층형 구조(Hierarchical Architecture)를 ViT에 적용**한 모델
- ViT의 한계: 모든 패치에 **동일한 크기·간격** 사용 → 물체 크기·해상도의 계층적 학습 어려움
- Swin vs CvT
  - Swin: 블록마다 **다른 로컬 윈도 크기**로 계층적 특징 학습
  - CvT: **CNN의 계층적 구조**를 ViT에 적용 → **지역 특징(Local) + 전역 특징(Global)** 모두 활용, **적은 매개변수**로 높은 성능
- 구조: **합성곱 계층**으로 지역 특징 추출 + **ViT 셀프 어텐션**으로 전역 특징 결합
  - Swin보다 **더 적은 매개변수**로 비슷한 수준의 성능
- 계층적 특징 예시 (얼굴 이미지)
  - **저수준(Low-level):** 선, 점
  - **중간 수준(Mid-level):** 눈, 귀
  - **고수준(High-level):** 얼굴 구조

ViT vs CvT

모델	위치 임베딩	패치 임베딩	어텐션 임베딩	계층형 구조
ViT	○	**겹치지 않는** 선형 임베딩	선형 임베딩	×
CvT	×	**겹치는(overlapping)** 합성곱 임베딩	합성곱 임베딩	○

- CvT는 **위치 임베딩 없이도** 성능 저하 없이 **다양한 해상도**의 이미지 처리 가능

1) 합성곱 토큰 임베딩 (Convolutional Token Embedding)

- **2D로 재구성된 토큰 맵**에서 **중첩 합성곱 연산**을 수행하는 임베딩 (ViT의 패치 임베딩과 문맥상 동일, 의역된 표현)
- ViT에 **계층적인 CNN의 성질**을 추가하기 위해 사용
- 기존 ViT: 선형 임베딩으로 Q, K, V 변환 → 이미지의 **2D 구조를 잃음**
- CvT: **2D 합성곱 임베딩**으로 Q, K, V 변환 → **2D 구조 보존**하면서 셀프 어텐션 계산
  - 예) 3×3 커널(K), 간격 1(S) → 합성곱 후 **1차원으로 재배열**하여 기존 셀프 어텐션 수행
- 효과: **지역 특징 학습**, **시퀀스 길이 ↓ + 토큰 특징 차원 ↑** (CNN의 다운 샘플링 + 특징 맵 수 증가와 유사)

2) 어텐션에 대한 합성곱 임베딩 (Convolutional Projection for Attention)

- 2D 토큰 맵에서 **분리 가능한 합성곱**을 적용해 **성능 저하 및 계산 복잡도를 낮추는** 연산
- 일반적으로 Q, K, V 차원은 동일하지만, CvT는 **K, V의 차원을 축소(Squeezed Convolutional Projection)**

구분	커널 크기(K)	간격(S)	생성 패치 수
쿼리(Q)	3×3	1	9개
키(K), 값(V)	3×3	2	4개

- Q(9), K(4), V(4) 길이가 달라도 **셀프 어텐션 결과 크기는 9×1로 동일**
  - Q(9×1) × K^T(1×4) → **어텐션 맵(9×4)**
  - 어텐션 맵(9×4) × V(4×1) → **셀프 어텐션 벡터(9×1)**
  - 입력·출력 벡터의 패치 길이가 동일, **계산할 패치 개수 ↓ → 연산량 ↓**
- 트랜스포머 계층이 깊어질수록 **패치 수 ↓, 벡터 차원 ↑** → 매개변수 수 대폭 감소
  - 예) 첫 블록 패치 64×64·임베딩 128 → 마지막 블록 패치 8×8(4배 감소)·임베딩 512(4배 증가)
  - 패치 크기가 줄어든 만큼 임베딩 차원을 늘려 **네트워크 안정성 ↑, 모델 설계 단순화**

3) 모델 실습 (CvT 미세 조정)

- 사전 학습 모델: `microsoft/cvt-21` (ImageNet-1k, 224×224)
- **ViT 실습과의 차이점**
  - 이미지 크기 전처리 시 `height/width`가 아닌 `shortest_edge` 키 사용 (너비·높이 중 **더 작은 값**, 모든 입력을 가장 짧은 길이로 정규화)
  - 모델 클래스 `CvtForImageClassification`

모델 구조

- 크게 **3개 스테이지(cvt)** + 계층 정규화 + 분류기
  - Swin과 달리 **풀링(pooler) 계층 없음** (이미 합성곱으로 지역·전역 정보를 파악하므로)
  - 스테이지별로 이미지 크기가 **4배, 8배, 16배 감소**, 반비례하게 임베딩 차원 ↑
- 각 스테이지 = **CvtEmbeddings** + **CvtLayer**
  - **CvtEmbeddings:** 2D 합성곱 + 계층 정규화 → 이미지를 패치 단위로 잘라 임베딩

        - 첫 스테이지: **7×7 커널, 4 간격, 2 패딩**
        - (224−7+2×2)/4+1 = 56.25 → 버림 → **56×56**, 출력 채널 64 → [N, 64, 56, 56]
    - **CvtLayer:** 3,136(56×56)개 패치에 대해 셀프 어텐션 수행
        - **키·값 임베딩의 간격을 2**로 설정 → 차원 축소 (쿼리는 간격 1)

단계	shape
입력 이미지	[32, 3, 224, 224]
패치 임베딩	[32, 64, 56, 56]
셀프 어텐션 입력	[32, 3136, 64]
셀프 어텐션 출력	[32, 3136, 64]

학습 결과 (CvT)

Epoch	Training Loss	Validation Loss	F1
1	0.7737	0.3269	0.8914
2	0.7008	0.2596	0.9172
3	0.6438	0.2460	0.9173

- 테스트 정확도 **약 91.3%**
- Swin 대비 F1 소폭 개선 (0.9159 → 0.9173)
- ViT 대비 **'셔츠' 카테고리 성능 개선**, Swin과 비슷한 성능

한눈에 정리

모델 비교

구분	ViT	Swin Transformer	CvT
핵심 아이디어	이미지를 패치로 나눠 트랜스포머에 입력	로컬 윈도 + 시프트 윈도	CNN의 계층 구조를 ViT에 적용
분류 방식	[CLS] 토큰	토큰 평균값	(풀링 없이) 분류기
위치 정보	위치 임베딩	상대적 위치 편향	위치 임베딩 × (합성곱)
패치 임베딩	겹치지 않는 선형 임베딩	선형 임베딩 + 패치 병합	겹치는 합성곱 임베딩
계층형 구조	×	○	○
어텐션 복잡도	2차	**1차** (윈도 단위)	K, V 축소로 감소
스테이지 수	-	4개	3개

성능 비교 (FashionMNIST)

구분	ViT	Swin Transformer	CvT
모델 크기	327,325 MB	105,227 MB	120,791 MB
F1-점수	**0.9231**	0.9159	0.9173
초당 처리 수	29.636 eval/s	**58.048 eval/s**	44.778 eval/s

- **F1-점수가 가장 높은 모델:** ViT

- **모델 크기 대비 가장 우수한 성능:** Swin Transformer
- **가장 일반화 성능이 좋았던 모델:** CvT
- 비전 트랜스포머 계열은 **작은 학습 데이터에서도 높은 정확도** → 분류 문제에 매우 유용, **문제·사용처에 따라 적절히 선택**